

Machine Learning Teaser — Predicting Customer Leaving

Introduction

Machine learning is a way of finding **patterns in data** so a computer can make predictions.

Instead of writing rules manually, we give the computer **examples of past data** and the algorithm learns relationships between variables.

Example:

A streaming service wants to know:

Which customers are likely to leave the service next month?

If we can predict this early, the company can send the customer a **discount or offer** to try to keep them.

This type of problem is called **supervised learning**.

In supervised learning:

- We have **input variables** describing something.
- We have a **target variable** describing the outcome.
- The algorithm learns how the inputs relate to the outcome.

Key Terms

Features

Features are the input variables used to make a prediction.

Example:

```
hours_watched_last_month  
days_since_last_login  
support_calls_last_month
```

These describe customer behaviour.

Target Variable

The **target variable** is what we want to predict.

In this exercise the target is:

```
will_leave
```

Values:

```
0 = customer stays  
1 = customer leaves
```

K-Nearest Neighbours (KNN)

In this exercise we use a simple algorithm called **K-Nearest Neighbours (KNN)**.

The idea is simple.

To classify a new customer:

1. Look at **similar customers** in the dataset.
2. Find the **K closest customers**.
3. See how many of them left the service.
4. Use the **majority vote** to make a prediction.

Example:

```
Nearest neighbours:  
Customer A → left  
Customer B → left  
Customer C → stayed
```

Prediction:

```
Customer will leave
```

Because **2 out of 3 neighbours left**.

Measuring Model Performance

Accuracy Score

Accuracy tells us **how often the model predicts correctly**.

Formula:

```
accuracy = correct predictions / total predictions
```

Example:

```
70 correct predictions out of 100  
accuracy = 70%
```

Confidence Score

Confidence shows **how strongly the neighbours support the prediction**.

Example:

```
K = 5 neighbours  
4 predict leaving  
1 predicts staying
```

Confidence:

```
4 / 5 = 0.80
```

Meaning **80% of neighbours agree**.

Scenario

A streaming service wants to predict whether a customer will **leave the service next month**.

If the model predicts the customer will leave, the company will send them a **special discount offer**.

You are given a dataset containing **1000 customers**.

Each row describes a customer's behaviour.

Your task is to build a **KNN prediction pipeline using dbt**.

Dataset

Table:

customers

Columns:

```
customer_id
signup_year
months_as_customer
monthly_fee
total_spent
hours_watched_last_month
days_since_last_login
support_calls_last_month
payment_failures_last_year
number_of_profiles
avg_watch_time_per_day
will_leave
```

Target column:

will_leave

Important Rule

You must choose **exactly FOUR** features that you believe influence whether a customer leaves.

Do **not** use the target variable as a feature.

What Has Been Provided

The project already contains:

Data

resources/customer_leaving.duckdb

Staging Model

```
stg_customers
```

This exposes the dataset.

Macro

A macro is provided in the `macros/` folder to calculate Euclidean distance:

```
macros/calculate_distance.sql
```

Use it in Step 5 to calculate distances between customers. It takes two table aliases as arguments:

```
{{ calculate_distance('t', 'tr') }} AS distance
```

This expands to the full Euclidean distance formula across all four normalised features.

Step 1 — Randomize the Dataset

Machine learning requires splitting data into:

- training data
- test data

Before splitting, we randomize the rows so the order does not influence the split.

Without randomisation, if customers happen to be ordered by signup year or `will_leave` status, the first 70% would not represent the full dataset — making the model appear more or less accurate than it really is.

Task

Complete the model:

```
models/intermediate/ml_randomize_customers.sql
```

Add a random value column and materialize the model as a table.

Step 2 — Create Training and Test Datasets

We train the model on part of the data and evaluate it on the rest.

Typical split:

```
70% training
30% testing
```

With 1000 customers, this gives ~700 training examples (enough to learn patterns) and ~300 test examples (enough to evaluate fairly). More training data produces a better model; more test data produces a fairer evaluation.

Tasks

Complete the models:

```
models/intermediate/ml_train_customers.sql
models/intermediate/ml_test_customers.sql
```

Filter based on the random value.

Step 3 — Select Your Features

Machine learning models depend on **choosing useful features**.

Some columns are helpful predictors. Some are not.

Task

Complete the model:

```
models/intermediate/ml_selected_features.sql
```

Select:

- exactly **4 features**
- the target column `will_leave`

Important: Alias your chosen features as `feature_1`, `feature_2`, `feature_3`, `feature_4`. The downstream normalisation, distance, and prediction models all depend on these exact column names.

Example structure:

```
SELECT
  customer_id,
  your_chosen_column_1 AS feature_1,
  your_chosen_column_2 AS feature_2,
  your_chosen_column_3 AS feature_3,
  your_chosen_column_4 AS feature_4,
  will_leave
FROM {{ ref('ml_train_customers') }}
```

Step 4 — Normalize Features

KNN works best when features are scaled to the same range.

KNN measures similarity using distance. If `monthly_fee` ranges 0–100 and `days_since_last_login` ranges 0–30, a small difference in `monthly_fee` dominates the distance calculation, effectively ignoring the other features. Min-Max scaling forces every feature into the range [0, 1] so all four contribute equally.

We use **Min-Max scaling**:

```
(value - min) / (max - min)
```

Implement this directly in your SQL using a CTE to calculate the min and max for each feature.

Tasks

Complete:

```
models/intermediate/ml_train_customers_n.sql
models/intermediate/ml_test_customers_n.sql
```

Normalize the selected features.

Important:

Test data must be scaled using **training dataset statistics**.

For `ml_test_customers_n`: the test table still has the original column names. Before normalising, apply the same feature aliases you chose in `ml_selected_features` to the test rows — a CTE works well for this. Both models must output: `customer_id`, `feature_1_n`, `feature_2_n`, `feature_3_n`, `feature_4_n`, `will_leave`.

Step 5 — Calculate Distances

KNN compares customers using **Euclidean distance**.

Distance formula:

```
sqrt((a1-b1)^2 + (a2-b2)^2 + ...)
```

Use the provided `calculate_distance` macro to calculate this distance. It takes two table aliases — one for the test customer and one for the training customer.

Example structure:

```
SELECT
  t.customer_id AS test_customer_id,
  tr.customer_id AS train_customer_id,
  tr.will_leave AS train_will_leave,
  {{ calculate_distance('t', 'tr') }} AS distance
FROM ...
```

Task

Complete:

```
models/intermediate/ml_test_distances.sql
```

Calculate distances between every test customer and every training customer.

Step 6 — Generate Predictions

For each test customer:

1. sort neighbours by distance
2. take the **K nearest neighbours**
3. predict using **majority vote**

Provided

This model is provided for you — review it to understand how KNN predictions are generated:

```
models/mart/ml_test_predictions.sql
```

Output:

```
test_customer_id
predicted_will_leave
confidence_score
```

K is the number of neighbours consulted for the vote. K = 5 is a sensible default for 1000 customers. Lower K (e.g. 3) is more sensitive to noise; higher K (e.g. 9) gives a smoother decision boundary. Use an odd number to avoid tie votes. You can change the K value in `models/mart/ml_k_value.sql`.

Step 7 — Measure Accuracy

Evaluate the model by comparing predictions with actual values.

Provided

This model is provided for you — review it to understand how accuracy is calculated:

```
models/mart/ml_accuracy_score.sql
```

Step 8 — Predict a New Customer

Predict whether the following customer will leave.

```
months_as_customer = 2
monthly_fee = 15
hours_watched_last_month = 3
days_since_last_login = 18
support_calls_last_month = 2
payment_failures_last_year = 1
number_of_profiles = 1
avg_watch_time_per_day = 0.1
```

Task

A prediction template is provided in:

```
models/mart/ml_prediction.sql
```

Open the file and replace each `NULL` with the correct value for **your chosen features** from the new customer data above. Four values to fill in — one per feature.

Output:

```
predicted_will_leave
confidence_score
```

Step 9 — Build and Inspect Your Pipeline

Once you have completed all models (Steps 1–8), run:

```
dbt run
```

dbt will build every model in dependency order. You should see all of the following complete successfully:

```
stg_customers
ml_randomize_customers
ml_train_customers      ml_test_customers
ml_selected_features
ml_train_customers_n    ml_test_customers_n
ml_test_distances
ml_k_value
ml_test_predictions
ml_accuracy_score
ml_prediction
```

Then run these two queries to inspect your results:

```
SELECT * FROM ml_accuracy_score;
SELECT * FROM ml_prediction;
```

Submission

Complete the answer sheet:

```
ml_teaser_answer_sheet.md
```

After `dbt run`, collect your numerical results with:

```
SELECT
  a.accuracy_score,
  p.predicted_will_leave,
  p.confidence_score
FROM      ml_accuracy_score a
CROSS JOIN ml_prediction    p;
```

Fill in the answer sheet with your results and written responses. Submit both your completed dbt models and the answer sheet.